FraudDetectML

Real-Time Fraud Detection Pipeline — Anomaly Detection, Graph Analytics & Streaming Risk Scoring

Tech Stack: Python · PyTorch · Kafka · Neo4j · FastAPI

Version v1.0 • June 2026 • Internal Engineering Specification

Table of Contents

1. Project Overview

FraudDetectML is a production-grade, real-time fraud detection system for fintech payment processing. It combines streaming anomaly detection, graph-based transaction analysis (Neo4j), and a PyTorch neural ensemble to score every transaction within 50 ms.

Language	Python 3.11+
ML Framework	PyTorch 2.3, PyTorch Geometric 2.5 (GNN)
Streaming	Apache Kafka 3.7 (KRaft mode, no ZooKeeper)
Graph DB	Neo4j 5.20 (Community / Enterprise)
API	FastAPI 0.111 with async handlers
Feature Store	Redis 7 (online) + Parquet on S3 (offline)
Target SLA	p99 scoring latency < 50 ms end-to-end
False Positive Rate Target	< 0.3% (to minimise customer friction)

1.1 Threat Model — Fraud Types Addressed

- **Card-present / CNP fraud:** stolen card credentials used for purchases
- **Account takeover (ATO):** attacker gains access to legitimate account
- **Synthetic identity fraud:** fabricated identities built from mixed PII fragments
- **Merchant collusion:** fraudulent merchants processing fake transactions
- **Money mule networks:** layered fund movement through connected accounts
- **Velocity attacks:** rapid succession of small transactions before detection

2. System Architecture

Component	Role	Technology
Payment Gateway	Produces transaction events to Kafka	External / Kafka Producer SDK
Feature Builder	Enriches transaction with velocity & graph features	Python, Redis, Neo4j
ML Scoring Engine	Runs anomaly + GNN + ensemble models	PyTorch, ONNX Runtime
Scoring API	Synchronous risk score endpoint for payment gateway	FastAPI, Redis
Graph Ingestion Worker	Writes transaction graph edges to Neo4j async	Kafka consumer, neo4j-driver
Alert Engine	Routes high-score transactions to review queue	Kafka, PostgreSQL
Case Manager UI	Fraud analyst review and feedback interface	React, FastAPI
Model Trainer	Offline training pipeline on labelled data	PyTorch, MLflow, S3

3. Data Streaming — Kafka

3.1 Topic Design

Topic	Producer	Consumer	Retention
txn.raw	Payment gateway	Feature builder, graph ingestor	7 days
txn.enriched	Feature builder	ML scoring engine	3 days
txn.scored	ML scoring engine	Alert engine, audit log	30 days
alerts.fraud	Alert engine	Case manager, block service	90 days
feedback.labels	Case manager (analyst decisions)	Model trainer (online learning)	365 days

3.2 Schema Registry

- All topic schemas registered in **Confluent Schema Registry** (Avro format)
- Schema evolution: BACKWARD_TRANSITIVE compatibility enforced for consumer safety
- Schema IDs embedded in message headers; deserialiser validates on consume

```
// Avro schema for txn.raw topic
# txn.raw Avro schema (abbreviated)
{"type":"record","name":"RawTransaction","fields":[
{"name":"txn_id","type":"string"},
{"name":"account_id","type":"string"},
{"name":"merchant_id","type":"string"},
{"name":"amount_usd","type":"double"},
{"name":"timestamp","type":{"type":"long","logicalType":"timestamp-millis"}},
{"name":"card_present","type":"boolean"},
{"name":"ip_address","type":["null","string"],"default":null},
{"name":"device_fingerprint","type":["null","string"],"default":null}
]}
```

4. Feature Engineering

4.1 Transaction-Level Features

- **amount_zscore:** z-score of transaction amount vs account 90-day rolling mean/std
- **hour_of_day, day_of_week:** temporal cyclical encoding (sin/cos)
- **merchant_category_code:** one-hot (top 50 MCCs; rest as 'other')
- **card_present:** binary flag
- **geo_velocity_kmh:** km/h implied by distance between last 2 transactions / time delta
- **cross_border:** binary flag if merchant country != card issuer country

4.2 Velocity Features (Redis Rolling Windows)

Feature	Window	Redis Key Pattern
txn_count_1h	1 hour	vel:{account_id}:cnt:1h
amount_sum_24h	24 hours	vel:{account_id}:sum:24h
unique_merchants_7d	7 days	vel:{account_id}:merchants:7d (HyperLogLog)
decline_count_1h	1 hour	vel:{account_id}:declines:1h
new_merchant_flag	90 days	vel:{account_id}:seen_merchants (Bloom filter)

4.3 Graph Features (Neo4j)

- **shared_device_accounts:** count of accounts sharing same device_fingerprint in last 30d
- **shared_ip_accounts:** count of accounts sharing IP in last 7d
- **pagerank_score:** PageRank of account node in transaction graph (weekly refresh)
- **community_id:** Louvain community label — accounts in known fraud communities scored higher
- **degree_centrality:** number of unique merchants transacted with in 30d

5. ML Models

5.1 Model Ensemble

Model	Type	Input	Output
Isolation Forest	Anomaly detection (unsupervised)	54 tabular features	Anomaly score [0,1]
XGBoost Classifier	Supervised binary classification	54 tabular + 5 graph features	P(fraud) [0,1]
GNN (GraphSAGE)	Graph-based supervised (PyTorch Geometric)	Account subgraph (2-hop)	P(fraud node) [0,1]
Ensemble Meta-Model	Logistic regression stacking	Outputs of above 3 models	Final risk score [0,1]

5.2 GNN Architecture — GraphSAGE

```
// GraphSAGE fraud detection model
# models/gnn.py
class FraudGNN(torch.nn.Module):
def __init__(self, in_channels=16, hidden=64, out_channels=1):
super().__init__()
self.conv1 = SAGEConv(in_channels, hidden)
self.conv2 = SAGEConv(hidden, hidden)
self.conv3 = SAGEConv(hidden, out_channels)
self.dropout = nn.Dropout(0.3)
def forward(self, x, edge_index):
x = F.relu(self.conv1(x, edge_index))
x = self.dropout(x)
x = F.relu(self.conv2(x, edge_index))
x = self.dropout(x)
return torch.sigmoid(self.conv3(x, edge_index))
```

5.3 Inference Optimisation

- Models exported to **ONNX** for runtime inference; 3–5× faster than native PyTorch on CPU
- ONNX models loaded into ONNX Runtime with **CUDAExecutionProvider** (GPU) or **CPUExecutionProvider**
- Model warm-up: 100 dummy inference calls at startup to JIT-compile execution graphs

- Batch scoring: Kafka consumer batches up to 50 transactions per ONNX call (max latency 20 ms)

6. Graph Database — Neo4j

6.1 Graph Schema

```
// Neo4j property graph schema
// Node labels
(:Account {account_id, created_at, risk_band})
(:Merchant {merchant_id, name, mcc, country})
(:Device {fingerprint, first_seen})
(:IPAddress {ip, asn, country})

// Relationship types
(:Account)-[:TRANSACTED_AT {txn_id, amount, timestamp, fraud_label}]->(:Merchant)
(:Account)-[:USED_DEVICE {first_seen, last_seen, count}]->(:Device)
(:Account)-[:CONNECTED_FROM {last_seen}]->(:IPAddress)
(:Account)-[:SHARED_DEVICE]->(:Account) // inferred, batch job
```

6.2 Key Cypher Queries

```
// Cypher queries
// Find accounts sharing device with a given account (2-hop)
MATCH (a:Account {account_id: $id})-[:USED_DEVICE]->(d:Device)
<-[:USED_DEVICE]-(b:Account)
WHERE b.account_id <> $id
RETURN b.account_id, b.risk_band, count(d) AS shared_devices
ORDER BY shared_devices DESC LIMIT 20;

// Louvain community detection (GDS)
CALL gds.louvain.write('fraud-graph', {writeProperty: 'community_id'})
YIELD communityCount, modularity;
```

7. Scoring API — FastAPI

7.1 Synchronous Scoring Endpoint

```
// FastAPI score endpoint
# api/routes/score.py
@router.post('/score', response_model=ScoreResponse)
async def score_transaction(req: ScoreRequest,
scorer: Scorer = Depends(get_scorer)):
# 1. Enrich with velocity features from Redis (~2ms)
features = await scorer.enrich(req.transaction)
# 2. Run ensemble model (~15ms on GPU, ~35ms on CPU)
result = await scorer.predict(features)
# 3. Async write to Kafka txn.scored topic (non-blocking)
asyncio.create_task(scorer.publish_score(req.transaction, result))
return ScoreResponse(
txn_id=req.transaction.txn_id,
risk_score=result.final_score,
decision=result.decision, # 'allow'|'review'|'block'
latency_ms=result.latency_ms
)
```

7.2 Decision Thresholds

Risk Score	Decision	Action
0.00 – 0.30	allow	Transaction proceeds immediately
0.30 – 0.70	review	Transaction proceeds; async case created for analyst
0.70 – 0.90	challenge	Step-up auth (OTP) required before proceeding
0.90 – 1.00	block	Transaction declined; account flagged; alert sent

8. Model Training & MLOps

8.1 Training Data

- Labelled dataset: 18 months of transactions with fraud labels from chargebacks and manual review
- Class imbalance: ~0.4% fraud rate; addressed with **SMOTE** oversampling and class-weight tuning
- Train/validation/test split: 70/15/15 by time (no future leakage)
- Features computed using same feature pipeline as production (enforced by shared library)

8.2 MLflow Experiment Tracking

- Every training run logged to MLflow: hyperparameters, metrics, confusion matrix, ONNX artifact
- Model registry with **Staging** → **Production** promotion workflow requiring champion/challenger A/B test
- Automated retraining triggered when: (a) weekly model performance audit shows AUROC drop >2pp, or (b) concept drift detected via Population Stability Index >0.2

8.3 Key Evaluation Metrics

Metric	Target	Rationale
AUROC	> 0.97	Overall discrimination power
Precision @ 90% Recall	> 60%	Catch most fraud; limit false positives
False Positive Rate	< 0.3%	Minimise customer friction
KS Statistic	> 0.75	Score separation between fraud/legit
Latency p99	< 50 ms	Production SLA for payment approval flow

9. Alert Management

9.1 Case Lifecycle

- **Auto-block:** score > 0.90 → transaction declined, account suspended, alert created
- **Review queue:** score 0.30–0.90 → case created in PostgreSQL **fraud_cases** table
- **Analyst review:** Case Manager UI shows transaction detail, feature breakdown, graph neighbours, model explainability (SHAP)
- **Disposition:** analyst marks as fraud / not fraud → label written to **feedback.labels** Kafka topic
- **Online learning:** labelled samples feed incremental model update (XGBoost only; GNN retrained nightly)

9.2 SHAP Explainability

```
// SHAP top-10 feature explanations per transaction
# explainability/shap_explainer.py
class FraudExplainer:
def __init__(self, xgb_model):
self.explainer = shap.TreeExplainer(xgb_model)
def explain(self, features: np.ndarray) -> list[dict]:
shap_vals = self.explainer.shap_values(features)[1] # fraud class
return sorted([
{'feature': FEATURE_NAMES[i], 'shap_value': float(v)}
for i, v in enumerate(shap_vals[0])
], key=lambda x: abs(x['shap_value']), reverse=True)[:10]
```

10. Security, Compliance & Fairness

10.1 PCI-DSS Requirements

- PAN (Primary Account Number) data tokenised before ingestion; raw PAN never stored or logged
- All inter-service communication over TLS 1.3 minimum; certificates rotated every 90 days
- Kafka topics containing payment data encrypted at rest (AES-256) and in transit
- Access to Neo4j and PostgreSQL restricted to service accounts via network policy; no direct developer access in production
- Quarterly penetration test and annual PCI QSA audit required before go-live

10.2 GDPR Data Handling

- IP addresses and device fingerprints treated as personal data; stored with TTL-based deletion (7-day IP, 90-day device)
- Right to erasure: account deletion triggers graph node anonymisation within 30 days
- Data processing agreements in place with all third-party model training data providers

10.3 Model Fairness Auditing

- Quarterly disparate impact analysis across demographic proxies (geography, age band, merchant category)
- False positive rates compared across subgroups; if any group >2× baseline FPR, model investigation triggered
- Fairness metrics logged to MLflow alongside standard performance metrics

■■ *FraudDetectML must be reviewed by a qualified PCI QSA and legal counsel specialising in payments regulation before processing live cardholder data. This document is an engineering specification, not a compliance certification.*